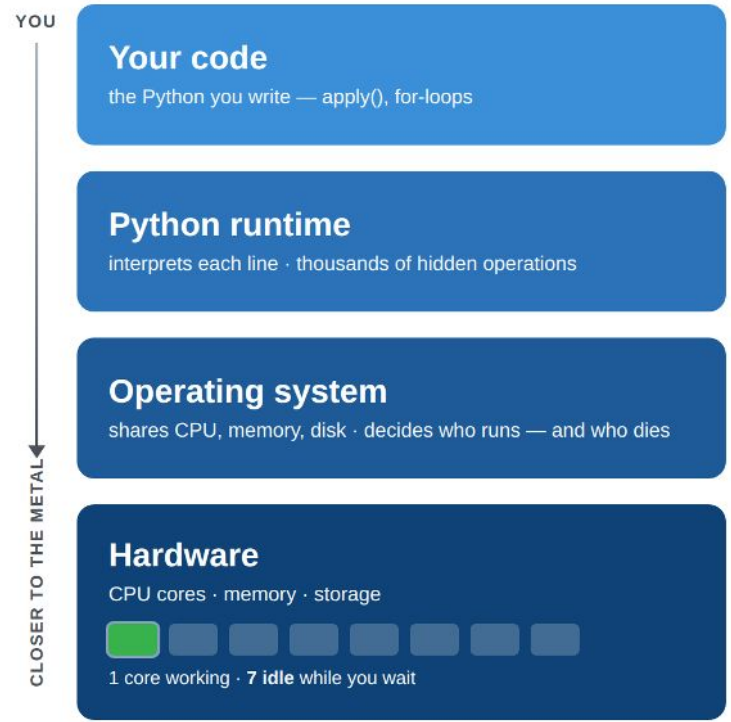
Operating Systems and Parallel Computing

Lecture 2

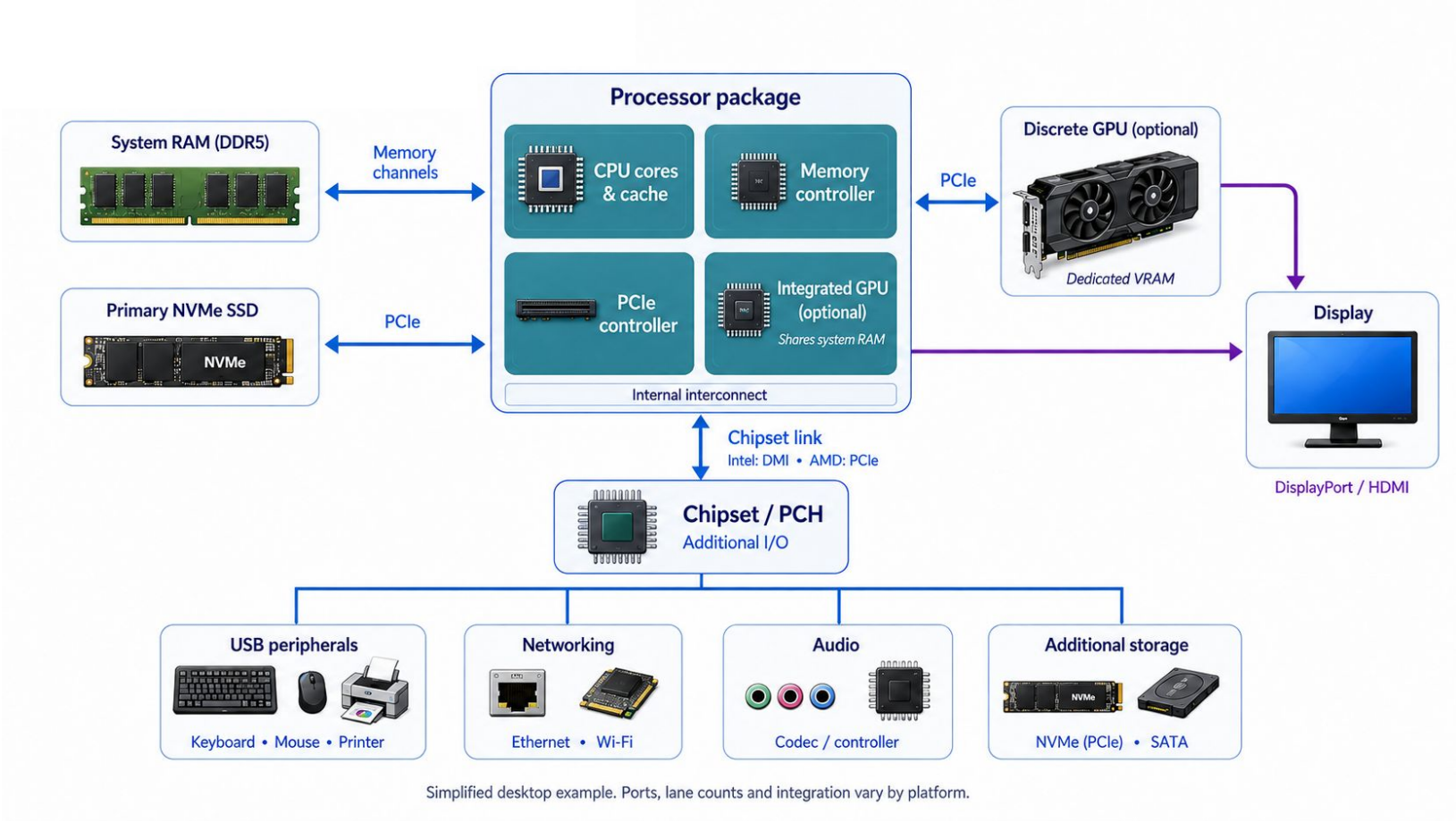
Prof. Matteo Turilli

Recap

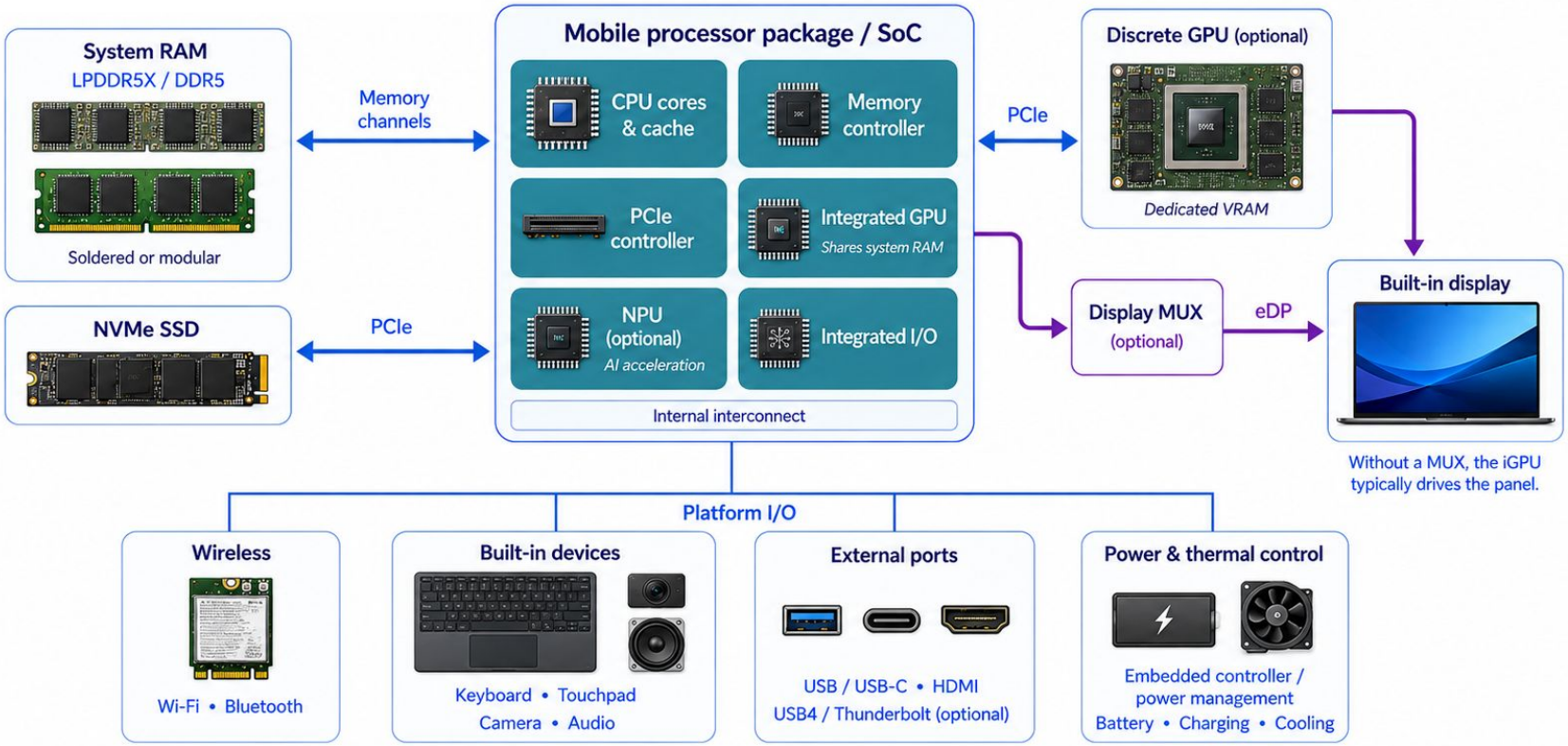
- We have a hardware platform:
  - CPU cores + often GPU cores/card
  - Random access memory (RAM)
  - Storage, e.g., hard disk (HD), solid state disk (SSD), non-volatile memory express (NVMe) disk
  - Network card(s), e.g., ethernet, wifi, bluetooth
- The operating system is the interface between the hardware and the software applications
- One of these applications is our Python runtime that interprets our Python programs
- Examples of other applications:
  - Word processors, e.g., MS Word
  - Spreadsheet, e.g., MS Excel
  - Web browser, e.g., Firefox, Chrome
  - Graphical user interface (GUI), e.g., windows, finder



Hardware: Modern Desktop Architecture

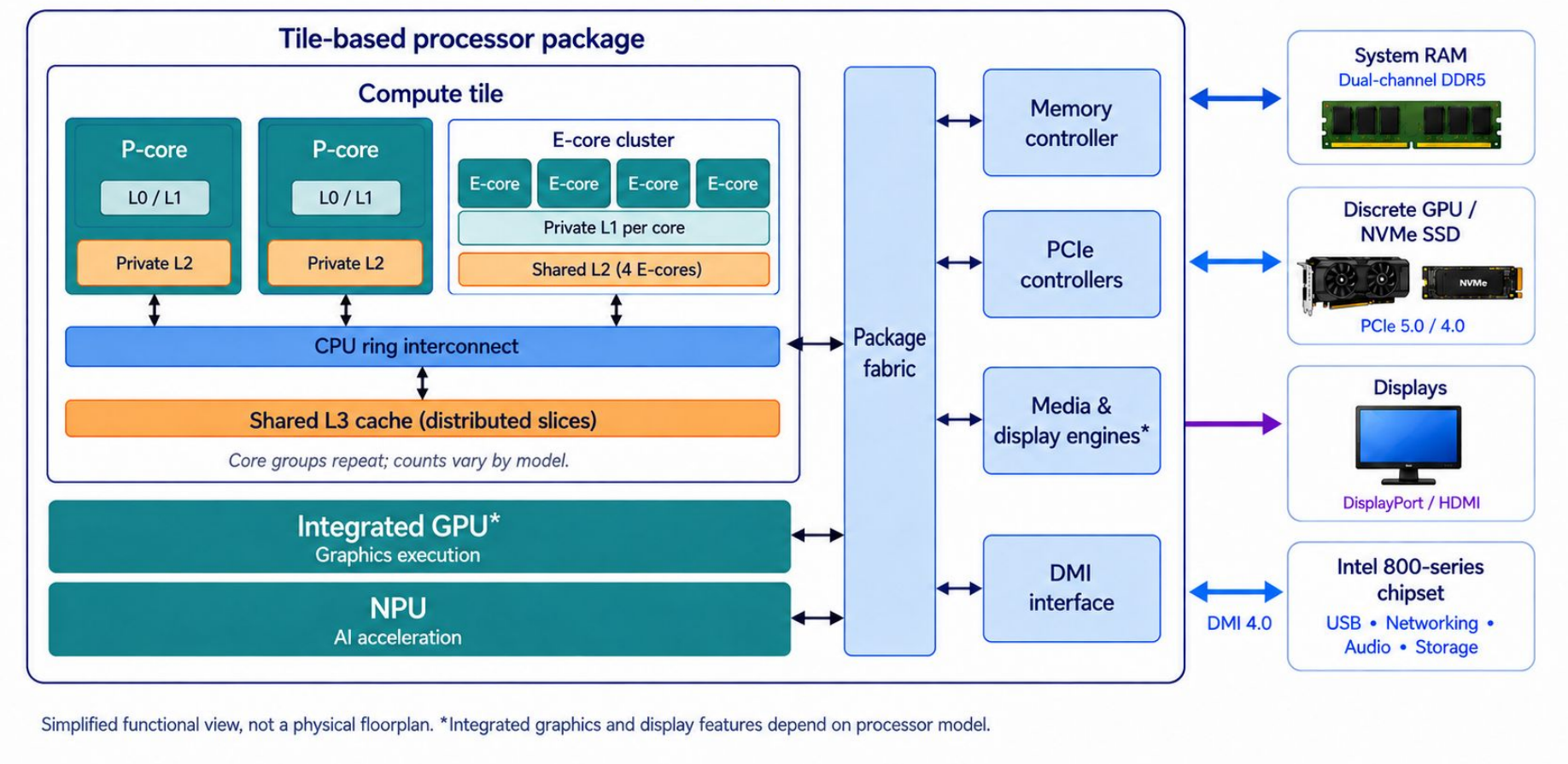


Hardware: Modern Laptop Architecture



Simplified mobile SoC example. Memory, I/O integration and display routing vary by model.

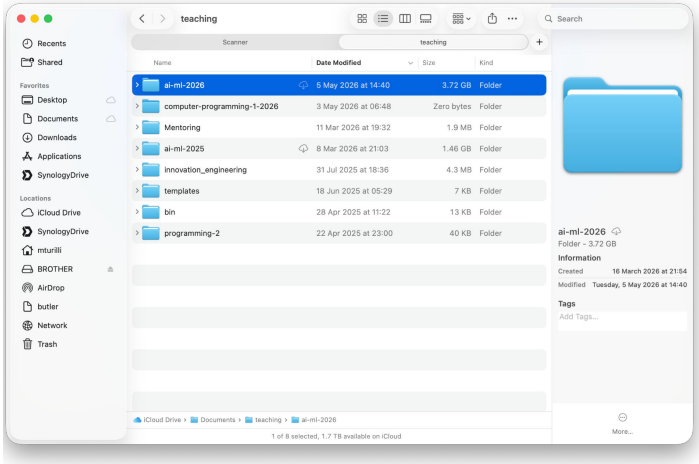
Hardware: Simplified Intel Arrow Lake Architecture (2024-)

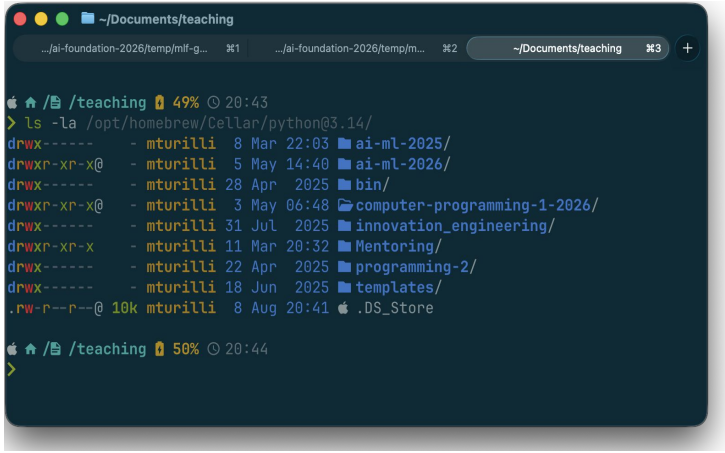


Simplified functional view, not a physical floorplan. *Integrated graphics and display features depend on processor model.

Why the Terminal?

- Servers, cloud instances, and our HPC cluster have no mouse interface for navigating windows.
- Every environment where production analytical computing happens is a Linux machine with an interface that requires typing and composing commands.
- The terminal is faster, scriptable, and reproducible: a command can be saved, shared, and re-run exactly.
- “Click here, then there” cannot be automated. A line of text that does the same operation can.
- By the end of today we will inspect a real dataset without ever opening a spreadsheet.





Filesystem Hierarchy Standard

- Linux (and other Unix OS) organize data following a standardized directory hierarchy.
- No volumes C, D, E, etc. Instead, directories that can be used as mount point for disk partitions
- Single tree that starts at the root, written /.
- Every program, configuration, data, even running processes are located somewhere under /.
- Our home is `/home/analyst`, also written `~`. It is where we store our files and we work.
- `data`: example dataset
- `tools`: example script to create a dataset
- `work`: **where we do all our work**. Mount point to the local filesystem, survives container reboots.

```
analyst@d0eb23d7399c:/$ tree -d -L 1 /
/
├── bin → usr/bin
├── boot
├── dev
├── etc
├── home
├── lib → usr/lib
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin → usr/sbin
├── srv
├── sys
├── tmp
├── usr
└── var
```

```
analyst@d0eb23d7399c:/$ tree -d -L 1 ~
/home/analyst
├── data
├── tools
└── work
```

Why the Terminal?

This is an interactive class

- Please run our course container and try out all the commands in the next slides
- Did you do your homework?! Do you remember how to run the container? ;-)

Why the Terminal?

This is an interactive class

- Please run our course container and try out all the commands in the next slides
- Did you do your homework?! Do you remember how to run the container? ;-)

```
docker compose run --rm ospc
```

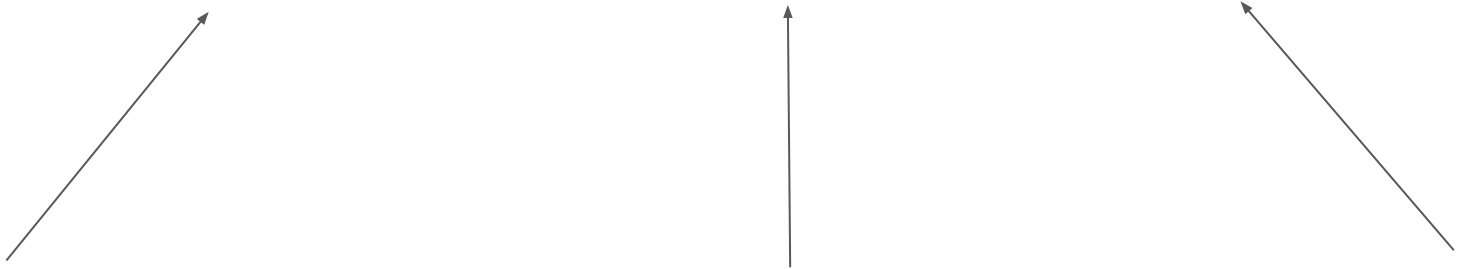
Navigating and Exploring the Tree

- `pwd`: prints current directory: “where am I in the tree?”
- `ls`: lists what is in the current directory
  - `ls -l` for detail
  - `ls -a` to show hidden files
- `cd`: change directory, moving around the tree
- Three important shortcuts:
  - `.` : current directory
  - `..` : parent directory
  - `~` : home directory
- Pro tips. Press:
  - `Tab` to autocomplete names,
  - `up-arrow` to recall previous commands.

```
analyst@d0eb23d7399c:~$ pwd
/home/analyst
analyst@d0eb23d7399c:~$ ls
data  tools  work
analyst@d0eb23d7399c:~$ ls -l
total 8
drwxr-xr-x  2 analyst analyst 4096 Aug  8 22:42 data
drwxr-xr-x  2 analyst analyst 4096 Aug  8 22:42 tools
drwx------+ 9 analyst analyst  288 Aug  9 00:37 work
analyst@d0eb23d7399c:~$ ls -a
.  ..  .bash_logout  .bashrc  .profile  data  tools
work
analyst@d0eb23d7399c:~$ cd /
analyst@d0eb23d7399c:/$ pwd
/
analyst@d0eb23d7399c:/$ ls
bin  boot  dev  etc  home  lib  media  mnt  opt  proc
  root  run  sbin  srv  sys  tmp  usr  var
```

Anatomy of a Command

```
ls -a -l -h data
```



Command | Options (flags) | Argument

- Read the command left to right: verb, adverbs, object
- Options can be combined: `-alh`
- Almost every command follows this shape.
- Once we recognize it, we have demystified the command line interface

Command Documentation

- How do we know the behavior or flags of a command? No one memorizes all flags.
- `man`: stays for ‘manual’, called with the name of the command.
  - E.g., `man ls`
- `–help`: concise version of man documentation when called with –help.
  - E.g., `ls –help`

```
LS(1)                                   General Commands Manual                                   LS(1)

NAME
       ls - List directory contents.  Ignore files and directories starting with a '.' by default

SYNOPSIS
       ls  [--help]  [--format]  [-C  ]  [-l|--long]  [-x ] [-T|--tabsize] [-m ] [--zero] [-D|--dired] [--hyperlink] [-1 ] [-o ] [-g ] [-n|--numeric-uid-gid]
       [--quoting-style] [-N|--literal] [-b|--escape] [-Q|--quote-name] [-q|--hide-control-chars]  [--show-control-chars]  [--time]  [-c  ]  [-u  ]  [--hide]
       [-I|--ignore] [-B|--ignore-backups] [--sort] [-S ] [-t ] [-v ] [-X ] [-U ] [-L|--dereference] [--dereference-command-line-symlink-to-dir] [-H|--deref-
       erence-command-line]  [-G|--no-group]  [--author]  [-a|--all]  [-A|--almost-all]  [-f ] [-d|--directory] [-h|--human-readable] [-k|--kibibytes] [--si]
       [--block-size] [-i|--inode] [-r|--reverse] [-R|--recursive] [-w|--width] [-s|--size] [--color] [--indicator-style] [-F|--classify] [--file-type] [-p ]
       [--time-style] [--full-time] [-Z|--context] [--group-directories-first] [-V|--version] [paths]

DESCRIPTION
       List directory contents.  Ignore files and directories starting with a '.' by default

OPTIONS
       --help Print help information.
```

Creating, Moving, Deleting Files and Directories

- `mkdir`: make a directory
  - `mkdir test_directory`
- `touch`: make an empty file
  - `touch test_file`
- `cp`: copy a source into a target (keeps the source)
  - `cp test_file test_file.bkp`
  - `cp -r test_directory bkp_directory`
  - `cp test_file.bkp test_directory/`
- `mv`: move a source to a target (deletes source)
  - `mv test_file.bkp bkp_directory/`
  - **Question**: what did happen to the `test_file` that was already in the `bkp_directory`?
- `rm`: remove a target
  - `rm test_directory/test_file.bkp`
  - `rm -r test_directory`

Working With Files

- `head`: read the first (n) rows of a file
  - `cd ~/data`
  - `head sales_2025_sample.csv` (by default it returns 10 lines)
  - `head -n 5 sales_2025_sample.csv`
- `tail`: read the last (n) rows of a file
  - `tail sales_2025_sample.csv`
  - `tail -n 5 sales_2025_sample.csv`
- `grep`: search for a string within file(s)
  - `grep Iberia sales_2025_sample.csv`
  - Grep is a very powerful tool, worth reading the man and tutorials
- `wc`: count the number of words in a file
  - `wc sales_2025_sample.csv` (man wc to understand the output)
- `less:` read a file
  - `less sales_2025_sample.csv` (arrows to move, q to quit)
  - Trivia: in unix the command was ‘more’, GNU made play on words, suggesting that "less" is "more" functional than "more."

Building a Data Science CL Toolkit

- `cut`: filters a column
- `sort`: orders rows
- `uniq`: filters duplicates

Read the man for these three commands and solve the following:

- Set the delimiter to ‘,’ and cut the third column from `sales_2025_sample.csv`
- Run `sort sales_2025_sample.csv` and observe the output:
  - How do we interpret the result? E.g., What column is sorted? How?
  - How do we use a human numeric sort?
  - What is the difference between the default sort and the human numeric one?
- Run `uniq sales_2025_sample.csv`. Did it do anything?

Building a Data Science CL Toolkit

- Extract the third column:
  - `cut -d ',' -f 3 sales_2025_sample.csv`
- Default sorting:
  - `sort sales_2025_sample.csv` compares entire lines as text, starting from the first character, not a selected CSV column.
  - Order follows the current locale; typically, 10 precedes 2.
- Human numeric sorting:
  - `sort -h sales_2025_sample.csv` interprets leading numbers and size suffixes such as K, M, and G: for example, 2K < 10K < 1M.
- Sort a specific numeric column:
  - `sort -t ',' -k3 sales_2025_sample.csv` sorts by column 3.

Building a Data Science CL Toolkit

- Remove duplicates:
  - `uniq sales_2025_sample.csv` removes only adjacent identical lines. If none are adjacent, the output is unchanged.
  - Use `sort sales_2025_sample.csv | uniq` to remove duplicates throughout the file.
- CSV caveats:
  - These commands treat the header as ordinary data; handle it separately when sorting.
  - `cut` does not understand quoted fields containing commas.

Terminal Editor

- We can edit files directly from the terminal
- Not a command but a terminal application
- Source of among the feistiest ‘religious wars’ in Unix history 8-D
- We introduce the `vi` editor because it is available in every unix. You can choose others, like `emacs` or `pico`.
- `vi` is a modal editor, i.e., it has different modes that need to be selected, depending on what we need to do. Here we see the editing and command modes.
- Hello_world in `vi`:
  a. We open `vi` issuing the `vi` command
  b. By default, we are in the command mode.
  c. We press `i` for input to switch to the editing mode.
  d. We write what we want
  e. We press `Esc` to switch back to the command mode
  f. We write `:w test_vi_file.txt` to write the file to disk
  g. We write `:q` to quit

Homework Before Next Session

- Make sure the course container works for you ask now if it does not. The rest of the course assumes you have it working.
- Redo today’s exercise from memory. Remember, fluency comes from repetition, not from reading.
- Goal: remember all the commands we covered with the main two of three flags.
- Keep the Terminal Cheat Sheet (on Blackboard) next to you while you practise.
- Next time: pipes and redirection, i.e., chaining these small tools into real data pipelines, and writing your first script.